

Web Services for Enterprise Application

**Unit-1
PG-Sem - III**

<https://www.w3.org/TR/ws-arch/>

<https://biet.ac.in/coursecontent/cse/IV-II%20CSE%20WSE&SOA.pdf>

Introduction: What are Web Services – Importance of web services – Web services and enterprises; XML Fundamentals:: XML Documents - Namespaces – Schema – Processing XML. Evolution and Emergence of Web Services: Evolution of distributed computing. Core distributed computing technologies: client/server, CORBA, JAVA RMI, Micro Soft DCOM, MOM, Challenges in Distributed Computing, basic operational model of web services, tools and technologies enabling web services, benefits and challenges of using web services.

What are Web Services ?

- A web service is a software module that is intended to carry out a specific set of functions. Web services in cloud computing can be found and invoked over the network. The web service would be able to deliver functionality to the client that invoked the web service.
- A web service is a set of open protocols and standards that allow data to be exchanged between different applications or systems. Web services can be used by software programs written in a variety of programming languages and running on a variety of platforms to exchange data via computer networks such as the Internet in a similar way to inter-process communication on a single computer.
- Any software, application, or cloud technology that uses standardized web protocols (HTTP or HTTPS) to connect, interoperate, and exchange data messages – commonly XML (Extensible Markup Language) – across the internet is considered a web service.

Web services have the advantage of allowing programs developed in different languages to connect with one another by exchanging data over a web service between clients and servers. A client invokes a web service by submitting an XML request, which the service responds with an XML response.

Functions of Web Services

- It's possible to access it via the internet or intranet networks.
- XML messaging protocol that is standardized.
- Operating system or programming language independent.
- Using the XML standard, it is self-describing.
- A simple location approach can be used to locate it.

Some of Real time Web Services

1. Payment Gateways

When you buy something online, the e-commerce site uses a web service to talk to a payment processor

PayPal: Securely processes transactions by connecting e-commerce platforms to banks and credit card systems.

Stripe: Provides APIs for handling one-time or recurring payments and invoicing

2. Social Media & Identity

These services allow websites to interact with social networks or verify your identity without a new password.

Social Logins: "Login with Google" or "Login with Facebook" buttons use authentication web services (often OAuth) to verify who you are.

Twitter/X API: Developers use this to display real-time tweets or post updates programmatically from other apps.

3. Real-Time Information

Web services power the live data you see in everyday mobile apps.

- **Weather Services:** Apps like **OpenWeatherMap** send your location to a server and receive the latest forecast data in return.

- **Google Maps API:** Used by apps like **Uber** or **Zomato** to provide live navigation, traffic updates, and location-based searching.

4. Travel & Logistics

Travel sites use web services to pull data from hundreds of different sources simultaneously.

- **Aggregators:** Sites like **Kayak** or **MakeMyTrip** communicate with airline and hotel web services to fetch real-time pricing and room availability.
- **Delivery Tracking:** Services used by apps like **DoorDash** or **Zomato** to track your food delivery in real-time.

5. Infrastructure & Enterprise

Large-scale cloud platforms offer hundreds of modular web services for developers.

- **Amazon Web Services (AWS):** Offers services like **S3** (for file storage) and **EC2** (for computing power) that companies like Netflix and Airbnb use to run their businesses.
- **GitHub API:** Allows other software to automatically interact with code repositories and manage technical issues.

Components of Web Service

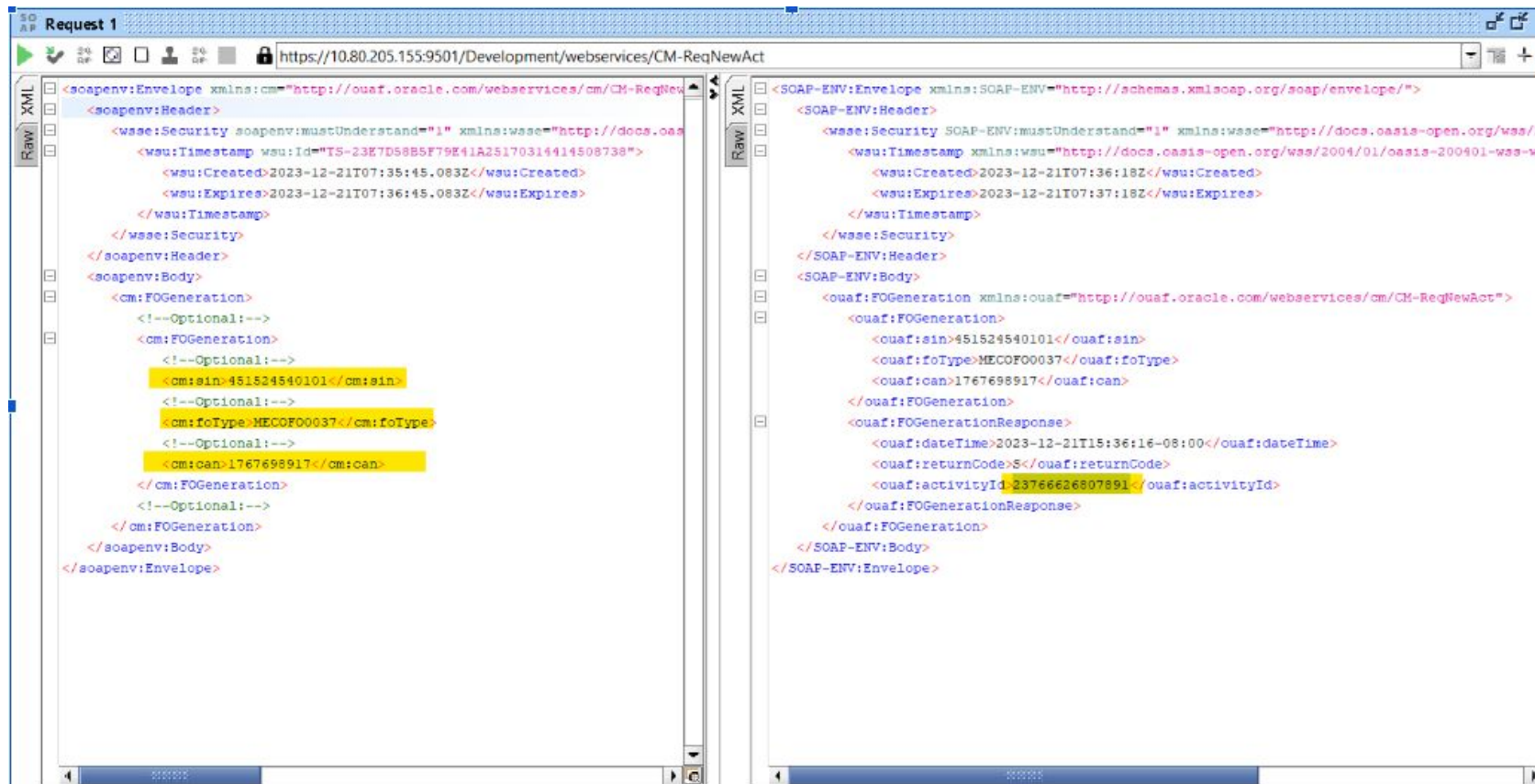
XML and HTTP is the most fundamental web services platform. The following components are used by all typical web services:

1. SOAP (Simple Object Access Protocol)

SOAP stands for "Simple Object Access Protocol." It is a transport-independent messaging protocol. SOAP is built on sending XML data in the form of SOAP Messages. A document known as an XML document is attached to each message. Only the structure of the XML document, not the content, follows a pattern. The best thing about Web services and SOAP is that everything is sent through HTTP, the standard web protocol.

A root element known as the envelope is required in every SOAP document. In an XML document, the root element is the first element. The "envelope" is separated into two halves. The header comes first, followed by the body. The routing data, or information that directs the XML document to which client it should be sent to, is contained in the header. The real message will be in the body.

Example code of SOAP UI



The **developer** or **architect of the web service** is fundamentally responsible for defining the request structure.

When you connect SoapUI to a service using its WSDL (Web Services Description Language), **SoapUI** auto-generates the sample request code. You (the tester or user) then fill in the specific data for that request

2. UDDI (Universal Description, Discovery, and Integration)

UDDI is a standard for specifying, publishing and discovering a service provider's online services. It provides a specification that aids in the hosting of data via web services. UDDI provides a repository where WSDL files can be hosted so that a client application can discover a WSDL file to learn about the various actions that a web service offers. As a result, the client application will have full access to the UDDI, which serves as a database for all WSDL files.

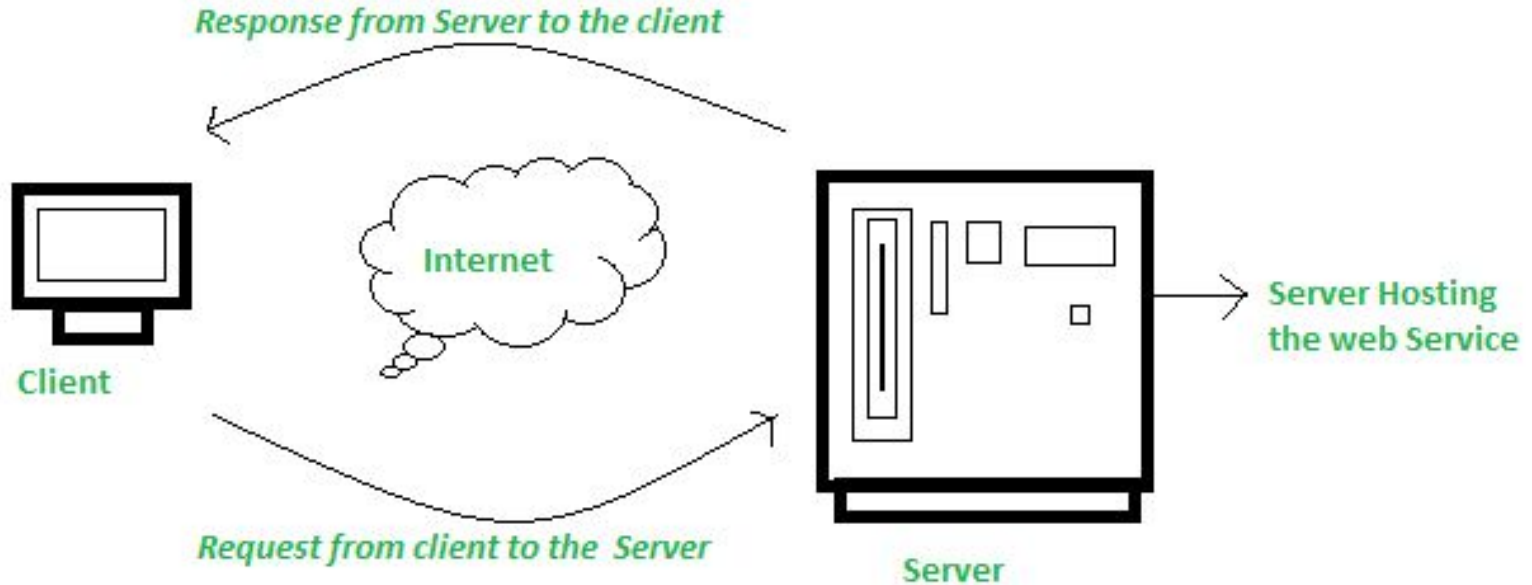
The UDDI registry will hold the required information for the online service, just like a telephone directory has the name, address, and phone number of a certain individual. So that a client application may figure out where it is.

3. WSDL (Web Services Description Language)

If a web service can't be found, it can't be used. The client invoking the web service should be aware of the location of the web service. Second, the client application must understand what the web service does in order to invoke the correct web service. The WSDL, or Web services description language, is used to accomplish this. The WSDL file is another XML-based file that explains what the web service does to the client application. The client application will be able to understand where the web service is located and how to use it by using the WSDL document.

How Does Web Service Work?

The diagram depicts a very simplified version of how a web service would function. The client would use requests to send a sequence of web service calls to a server that would host the actual web service.



- Remote procedure calls are what are used to make these requests. Calls to methods hosted by the relevant web service are known as Remote Procedure Calls (RPC). Example: Flipkart offers a web service that displays prices for items offered on Flipkart.com. The front end or presentation layer can be written in .Net or Java, but the web service can be communicated using either programming language.
- The data that is exchanged between the client and the server, which is XML, is the most important part of a web service design. XML (Extensible markup language) is a simple intermediate language that is understood by various programming languages. It is a counterpart to HTML. As a result, when programs communicate with one another, they do so using XML. This creates a common platform for applications written in different programming languages to communicate with one another.
- For transmitting XML data between applications, web services employ SOAP (Simple Object Access Protocol). The data is sent using standard HTTP. A SOAP message is data that is sent from the web service to the application. An XML document is all that is contained in a SOAP message. The client application that calls the web service can be created in any programming language because the content is written in XML.

Features/Characteristics Of Web Service

Web services have the following features:

(a) XML Based: The information representation and record transportation layers of a web service employ XML. There is no need for networking, operating system, or platform binding when using XML. At the middle level, web offering-based applications are highly interoperable.

(b) Loosely Coupled: A customer of an internet service provider isn't necessarily directly linked to that service provider. The user interface for a web service provider can change over time without impacting the user's ability to interact with the service provider. A strongly coupled system means that the patron's and server's decisions are inextricably linked, indicating that if one interface changes, the other should be updated as well.

A loosely connected architecture makes software systems more manageable and allows for easier integration between different structures.

(c) Capability to be Synchronous or Asynchronous: Synchronicity refers to the client's connection to the function's execution. The client is blocked and the client has to wait for the service to complete its operation, before continuing in synchronous invocations. Asynchronous operations allow a client to invoke a task and then continue with other tasks. Asynchronous clients get their results later, but synchronous clients get their effect immediately when the service is completed. The ability to enable loosely linked systems requires asynchronous capabilities.

(d) Coarse-Grained: Object-oriented systems, such as Java, make their services available through individual methods. At the corporate level, a character technique is far too fine an operation to be useful. Building a Java application from the ground, necessitates the development of several fine-grained strategies, which are then combined into a rough-grained provider that is consumed by either a buyer or a service. Corporations should be coarse-grained, as should the interfaces they expose. Web services generation is an easy approach to define coarse-grained services that have access to enough commercial enterprise logic.

(e) Supports Remote Procedure Call: Consumers can use an XML-based protocol to call procedures, functions, and methods on remote objects utilizing web services. A web service must support the input and output framework exposed by remote systems. Enterprise-wide component development Over the last few years, JavaBeans (EJBs) and .NET Components have become more prevalent in architectural and enterprise deployments. A number of RPC techniques are used to allocate and access both technologies. A web function can support RPC by offering its own services, similar to those of a traditional role, or by translating incoming invocations into an EJB or .NET component invocation.

(f) Supports Document Exchanges: One of XML's most appealing features is its simple approach to communicating with data and complex entities. These records can be as simple as talking to a current address or as complex as talking to an entire book or a Request for Quotation. Web administrations facilitate the simple exchange of archives, which aids incorporate reconciliation. The web benefit design can be seen in two ways: **(i)** The first step is to examine each web benefit on-screen character in detail. **(ii)** The second is to take a look at the rapidly growing web benefit convention stack.

Advantages Of Web Service

Using web services has the following advantages:

(a) Business Functions can be exposed over the Internet: A web service is a controlled code component that delivers functionality to client applications or end-users. This capability can be accessed over the HTTP protocol, which means it can be accessed from anywhere on the internet. Because all apps are now accessible via the internet, Web services have become increasingly valuable. Because all apps are now accessible via the internet, Web services have become increasingly valuable. That is to say, the web service can be located anywhere on the internet and provide the required functionality.

(b) Interoperability: Web administrations allow diverse apps to communicate with one another and exchange information and services. Different apps can also make use of web services. A .NET application, for example, can communicate with Java web administrations and vice versa. To make the application stage and innovation self-contained, web administrations are used.

(c) Communication with Low Cost: Because web services employ the SOAP over HTTP protocol, you can use your existing low-cost internet connection to implement them. Web services can be developed using additional dependable transport protocols, such as FTP, in addition to SOAP over HTTP.

(d) A Standard Protocol that Everyone Understands: Web services communicate via a defined industry protocol. In the web services protocol stack, all four layers (Service Transport, XML Messaging, Service Description, and Service Discovery) use well-defined protocols.

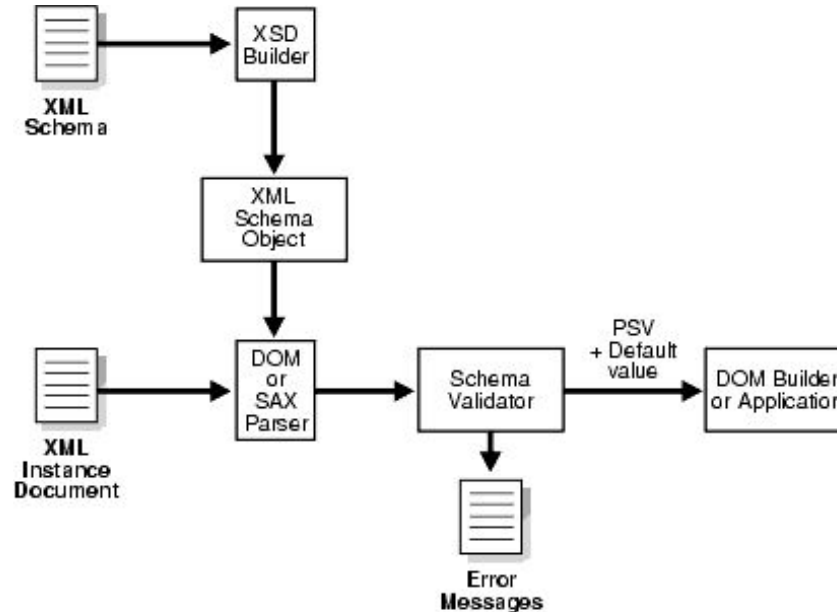
(e) Reusability: A single web service can be used simultaneously by several client applications.

XML Fundamentals:: XML Documents - Namespaces – Schema – Processing XML

<https://ronitbanerjee.com/xml-and-schema>

https://developer.mozilla.org/en-US/docs/Web/XML/Guides/XML_introduction

XML (eXtensible Markup Language) and XML Schema are foundational technologies for the Semantic Web, and understanding them is crucial for working with more advanced Semantic Web technologies



XML, or Extensible Markup Language, is a markup language similar to HTML. However, unlike HTML which has predefined tags, XML allows you to define your own tags tailored to your specific needs. This makes XML a powerful tool for storing data in a format that can be stored, searched, and shared. The fundamental format of XML is standardized, which means if you share or transmit XML across systems or platforms, the recipient can still parse the data due to the standardized XML syntax.

XML is the basis for many languages, including XHTML, MathML, SVG, RSS, and RDF. You can also define your own XML-based languages. The structure of an XML document is built on tags, with the XML declaration used for the transmission of the meta-data of a document.

For an XML document to be correct, it must be well-formed and conform to all XML syntax rules. It must also conform to semantic rules, which are usually set in an XML schema or a DTD (Document Type Definition).

XML also offers methods (called entities) for referring to some special reserved characters. There are five of these characters that you should know:

- `<` for the less than sign (`<`)
- `>` for the greater than sign (`>`)
- `&` for the ampersand (`&`)
- `"` for one double-quotation mark (`"`)
- `'` for one apostrophe (or single-quotation mark) (`'`)

XML is usually used for descriptive purposes, but there are ways to display XML data. If you don't define a specific way for the XML to be rendered, the raw XML is displayed in the browser. One way to style XML output is to specify CSS to apply to the document using the `xml-stylesheet` processing instruction. There is also another more powerful way to display XML: the Extensible Stylesheet Language Transformations (XSLT) which can be used to transform XML into other languages such as HTML. This makes XML incredibly versatile.

XML declaration

Namespace declaration for XML Schema

```
<?xml version="1.0" encoding="UTF-8"?>  
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"  
    ..... ">  
    <xs:element ...>  
        .....  
    </xs:element>
```

The diagram illustrates the structure of an XML Schema (XSD) declaration. It shows a sequence of XML elements: an XML declaration, an XSD schema element with a namespace declaration, and an XSD element. Arrows point from descriptive labels to these elements: 'XML declaration' points to the first line, 'Namespace declaration for XML Schema' points to the 'xmlns:xs' attribute in the second line, and 'XSD root element' points to the 'xs:element' tag in the third line.

XSD root element

XML is often used for data storage and transport. Here are some basic components of XML syntax:

Elements: XML documents contain elements, defined by a start tag and an end tag. For example, `<name>John Doe</name>`. Elements can contain text, other elements, or be empty.

Attributes: Elements can have attributes, which are name-value pairs. For example, `<person id="1">John Doe</person>`. Here, `id` is an attribute of the `person` element.

Namespaces: XML namespaces are used to avoid element name conflicts. If two different XML-based languages or vocabularies have elements or attributes with the same name, namespaces can be used to distinguish them.

```
<?xml version="1.0" encoding="UTF-8"?>
  <employees
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="employee.xsd">

    <employee id="101">
      <name>Alex Rodriguez</name>
      <department>Engineering</department>
      <email>alex@company.com</email>
      <join_date>2023-05-14</join_date>
    </employee>

    <employee id="102">
      <name>Sarah Chen</name>
      <department>Marketing</department>
      <email>sarah@company.com</email>
      <join_date>2024-11-22</join_date>
    </employee>

  </employees>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="employees">
    <xs:complexType>
      <xs:sequence>

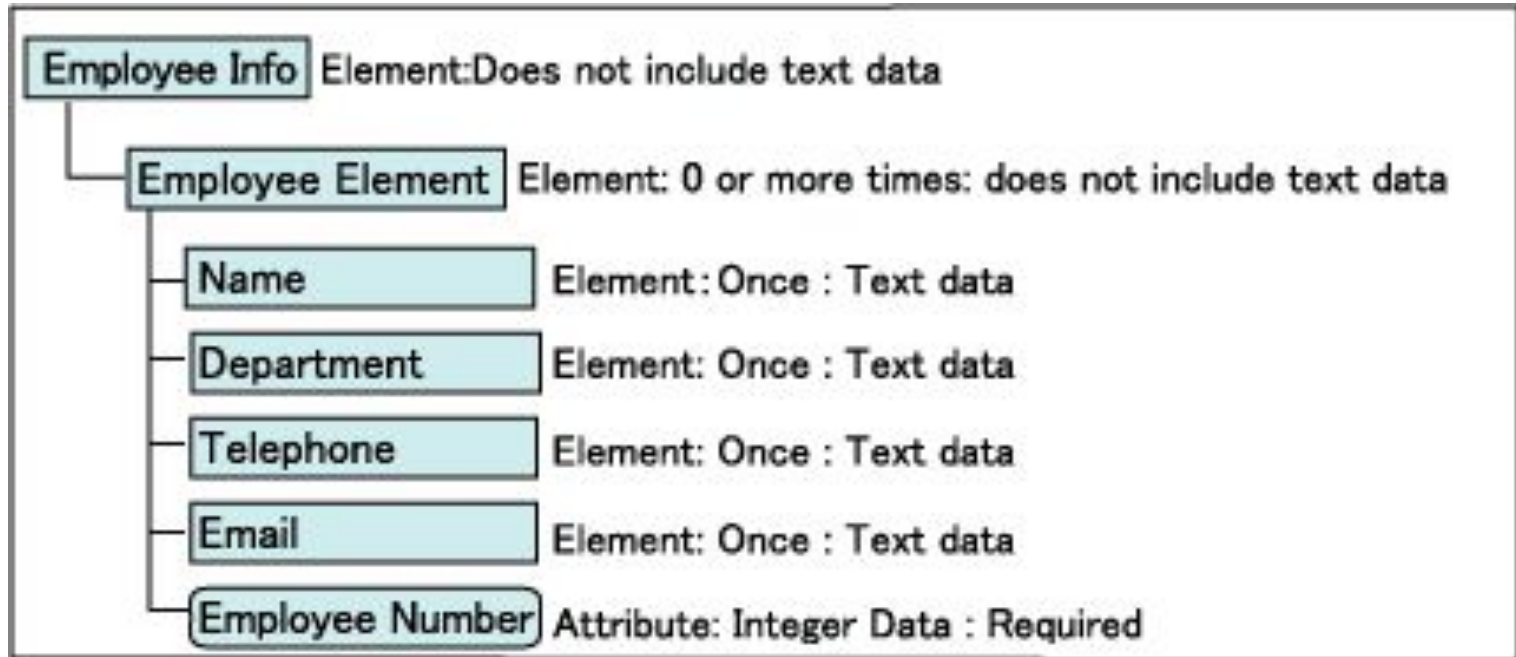
        <xs:element name="employee" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="department" type="xs:string"/>
              <xs:element name="email" type="xs:string"/>
              <xs:element name="join_date" type="xs:date"/>
            </xs:sequence>
            <xs:attribute name="id" type="xs:integer" use="required"/>
          </xs:complexType>
        </xs:element>

      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

`breakfast_menu` is an element that contains `food` elements, each of which has `name`, `price`, `description`, and `calories` elements.

XML Trees and DOM



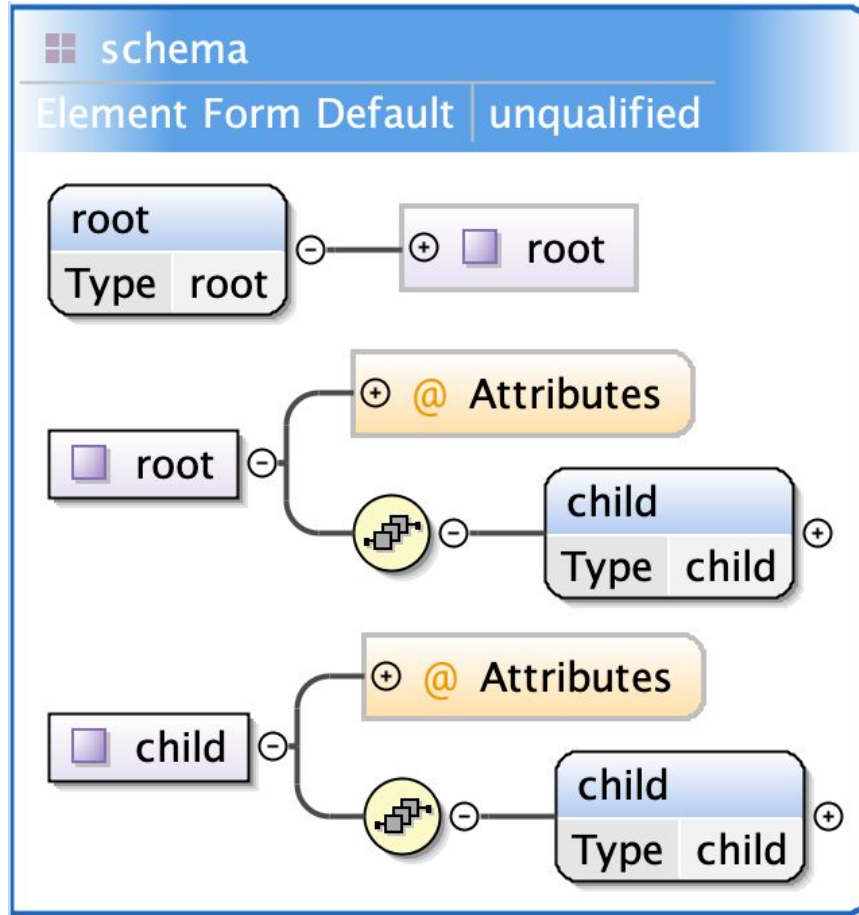
XML, or Extensible Markup Language, is a markup language used to encode documents in a format that is both human-readable and machine-readable. It is designed to store and transport data. XML documents form a tree structure that starts at "the root" and branches to "the leaves". Each element in the XML document can have child elements, attributes, and text.

The Document Object Model (DOM) is a programming interface for XML and HTML documents. It represents the structure of a document and allows programs and scripts to manipulate the document's structure, style, and content. The DOM represents a document as a tree structure, where each node in the tree corresponds to a part of the document (such as an XML or HTML tag).

XML Namespaces

- XML namespaces are used to avoid name conflicts in XML documents. They are essential for two primary reasons: to disambiguate between two elements that share the same name and to group elements relating to a common idea together. For example, in (x)html, there is a table element, and there is also an element of the same name in XSL-FO. Similarly, 'a', 'title', and 'style' are all elements in both (x)html and SVG. XML namespaces help distinguish between these elements that share the same name but belong to different XML vocabularies.
- An XML namespace is a unique URI (Uniform Resource Identifier). In an XML document, the URI is associated with a prefix, and this prefix is used with each element to indicate to which namespace the element belongs. For example, 'rdf:description', 'xsl:template', 'zblsa:data' are all examples of elements with namespace prefixes. The part before the colon is the prefix, the part after the colon is the local part, any prefixed element is a qualified name, and any un-prefixed element is an unqualified name.
- To use a namespace, you first associate the URI with a namespace. This is done using the 'xmlns' attribute. For example, '`<foo:tag xmlns:foo=\"[http://me.com/namespaces/foofoo\"](http://me.com/namespaces/foofoo)\">`' defines 'foo' as the prefix for the namespace for that element tag. The attribute prefixed with 'xmlns' works like a command to say "link the following letters to a URI". As no well-formed document can contain two identical attributes, the part that appears after the colon stops the same prefix being defined twice simultaneously.

XML Processing



Java:

Oracle provides a comprehensive tutorial on Java API for XML Processing (JAXP) which allows you to parse, transform, validate and query XML documents using Java. You can find the tutorial here: [Java API for XML Processing (JAXP) Tutorial](<https://docs.oracle.com/javase/tutorial/jaxp/TOC.html>).

XML Schema

XML Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="employee">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="empname" type="xs:string"/>
        <xs:element name="company" type="xs:string"/>
        <xs:element name="contactno" type="xs:int"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



XML Schema, often referred to as XML Schema Definition (XSD), is a language used to describe the structure of an XML document. It defines the legal building blocks of an XML document, including the elements and attributes that can appear, the number and order of child elements, the data types for elements and attributes, and the default and fixed values for elements and attributes.

Here is an example of an XML Schema:

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="note">

    <xs:complexType>

      <xs:sequence>

        <xs:element name="to" type="xs:string"></xs:element>

        <xs:element name="from" type="xs:string"></xs:element>

        <xs:element name="heading" type="xs:string"></xs:element>

        <xs:element name="body" type="xs:string"></xs:element>

      </xs:sequence>

    </xs:complexType>

  </xs:element>

</xs:schema>
```

- You can use an XML editor to edit the Schema files, an XML parser to parse the Schema files, and the Schemas can be manipulated with the XML DOM. They can also be transformed with XSLT.
- XML Schemas are extensible, meaning your Schema can be reused in other Schemas, you can create your own data types derived from the standard types, and multiple schemas can be referenced in the same document.
- XML Schemas also play a crucial role in securing data communication. They ensure that both sender and receiver have the same "expectations" about the content, which is particularly important when sending data from a sender to a receiver.
- For example, the date "06-08-2020" could be interpreted either as 6 August or as 8 June. However, an XML element with a data type like this: `<date type="date">2020-08-06</date>` ensures a mutual understanding of the content because the XML data type "date" requires the format "YYYY-MM-DD".

XML Schema Elements:

XML Schema elements are the building blocks of an XML document. They are defined in the XML Schema and can include complex types, which are a combination of elements and attributes. For example, in the provided XSD example, the "note" element is a complex type that includes a sequence of four elements: "to", "from", "heading", and "body". Each of these elements is defined as a string data type.

XML Schema Attributes:

XML Schema attributes provide additional information about XML elements. They are defined in the XML Schema and can be used to specify data types, default values, and fixed values for elements. Attributes are typically used to provide information that is not part of the main data content, such as identifiers, names, or characteristics of elements

XML Schema Structures:

XML Schema structures define the organization of elements and attributes in an XML document. They include sequences, choices, and all groups. A sequence is a group of elements that must appear in a specific order. A choice is a group of elements where only one can appear. An all group is a group of elements where all elements must appear, but the order is not important. These structures allow for more complex and flexible XML document designs.

Built-in datatypes in XML Schema include string, decimal, integer, boolean, date, and time, among others. You can also define your own simple types. For example, you might define a simple type that restricts strings to a certain pattern or restricts integers to a certain range.

XML Processing refers to the techniques and tools used to read, parse, validate, manipulate, transform, and generate XML documents.

XML processing is essential for exchanging and managing structured data between different applications and platforms.

Steps in XML Processing

1. **Parsing XML**

- Reading an XML document and converting it into a format that a program can understand.
- The parser checks whether the XML document is well-formed.

2. **Validation**

- Ensures that the XML document follows a predefined structure using:
 - DTD (Document Type Definition)
 - XSD (XML Schema Definition)

3. **Data Access and Manipulation**

- Extracting, updating, adding, or deleting XML elements and attributes.

4. **Transformation**

- Converting XML data into another format such as:
 - HTML
 - PDF
 - Another XML structure
- Usually done using XSLT (Extensible Stylesheet Language Transformations).

5. **Generation**

- Creating XML documents programmatically from application data.

XML Parsing

Student.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student>
    <id>101</id>
    <name>Monika</name>
    <course>MCA</course>
  </student>
  <student>
    <id>102</id>
    <name>Rahul</name>
    <course>BCA</course>
  </student>
</students>
```

XMLParserDemo

```
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;
import java.io.File;

public class XMLParserDemo {
    public static void main(String[] args) {
        try {
            File xmlFile = new File("student.xml");
            DocumentBuilderFactory factory =
                DocumentBuilderFactory.newInstance();
            DocumentBuilder builder =
                factory.newDocumentBuilder();
            Document document =
                builder.parse(xmlFile);
            document.getDocumentElement().normalize();
            System.out.println("Root Element : "
                +
                document.getDocumentElement().getNodeName());
            NodeList list =
                document.getElementsByTagName("student");
```

```
        for (int i = 0; i < list.getLength(); i++) {
            Element element = (Element) list.item(i);
            String id = element
                .getElementsByTagName("id")
                .item(0)
                .getTextContent();
            String name = element
                .getElementsByTagName("name")
                .item(0)
                .getTextContent();
            String course = element
                .getElementsByTagName("course")
                .item(0)
                .getTextContent();
            System.out.println("\nStudent " + (i + 1));
            System.out.println("ID : " + id);
            System.out.println("Name : " + name);
            System.out.println("Course : " + course);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: demoxml, FirstWS, RMIExample, Servers, SOAPDemo, xmlExample, and XMLParsing. Under XMLParsing, there is a JRE System Library [JavaSE-1.8], a src folder, and a default package containing XMLParserDemo.java and student.xml.

The main editor displays the XMLParserDemo.java file with the following code:

```
1 import javax.xml.parsers.DocumentBuilder;
2 import javax.xml.parsers.DocumentBuilderFactory;
3
4 import org.w3c.dom.Document;
5 import org.w3c.dom.Element;
6 import org.w3c.dom.NodeList;
7
8 import java.io.File;
9
10 public class XMLParserDemo {
11
12     public static void main(String[] args) {
13
14         try {
15
16             File xmlFile = new File("student.xml");
17
18             DocumentBuilderFactory factory =
```

The Console window at the bottom shows the output of the application:

```
<terminated> XMLParserDemo [Java Application] C:\Users\monikatiwari.INDUS\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe
Root Element : students

Student 1
ID : 101
Name : Monika
Course : MCA

Student 2
ID : 102
Name : Rahul
Course : BCA
```

XML Example of Testing with SOAP UI

The screenshot displays the SoapUI 5.9.1 application window. The main interface shows a SOAP request and response for the endpoint `http://www.dneonline.com/calculator.asmx`. The request is a SOAP envelope with a header and a body containing an `Add` operation. The response is a SOAP envelope with a header and a body containing an `AddResult` element with the value `30`.

A "New SOAP Project" dialog box is open in the center of the screen. The dialog has the following fields and options:

- Project Name:** (empty text field)
- Initial WSDL:** (empty text field with a "Browse..." button)
- Create Requests:** ☒ Create sample requests for all operations?
- Create TestSuite:** ☐ Creates a TestSuite for the imported WSDL
- Relative Paths:** ☐ Stores all file paths in project relatively to project file (requires save)
- Buttons:** OK, Cancel

The background interface includes a Navigator on the left showing a project tree with folders like "Demo", "FreeWSDL", "CalculatorSoap", and "CalculatorSoap12". The bottom of the window shows a status bar with various logs (SoapUI log, http log, jetty log, error log, wsm log, memory log) and a system tray with the date and time (23:37, 27-05-2026).

SOAP Testing using Java application

A Calculator SOAP Service with operations:

- add
- subtract

STEP 1: Create Java Project in Eclipse - SOAPCalculator

STEP 2: Create Package - com.soap

STEP 3: Create Service Interface - CalculatorService

```
package com.soap;
```

```
import javax.jws.WebMethod;  
import javax.jws.WebService;
```

```
@WebService  
public interface CalculatorService {
```

```
    @WebMethod  
    int add(int a, int b);
```

```
    @WebMethod  
    int subtract(int a, int b);  
}
```

STEP 4: Create Service Implementation - CalculatorServiceImpl

```
package com.soap;
```

```
import javax.jws.WebService;
```

```
@WebService(endpointInterface = "com.soap.CalculatorService")  
public class CalculatorServiceImpl implements CalculatorService {
```

```
    @Override  
    public int add(int a, int b) {  
        return a + b;  
    }
```

```
    @Override  
    public int subtract(int a, int b) {  
        return a - b;  
    }  
}
```

STEP 5: Publish SOAP Service - Publisher.java

```
package com.soap;

import javax.xml.ws.Endpoint;

public class Publisher {

    public static void main(String[] args) {

        Endpoint.publish(
            "http://localhost:8080/calculator",
            new CalculatorServiceImpl()
        );

        System.out.println("SOAP Service Started!");
    }
}
```


STEP 6: Run Publisher.java

On Console : SOAP Service Started!

STEP 7: Open Generated WSDL -

<http://localhost:8080/calculator?wsdl>

Generated WSDL Example - <wsdl:definitions ...>

<wsdl:service name="CalculatorServiceImplService">

<wsdl:port name="CalculatorServiceImplPort">

STEP 8: Test in SOAP UI

Create SOAP Project

STEP 9: Test Add Operation

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:com="http://soap.com/">
  <soapenv:Header/>
  <soapenv:Body>
    <com:add>
      <arg0>10</arg0>
      <arg1>20</arg1>
    </com:add>
  </soapenv:Body>
</soapenv:Envelope>
```

Expected Response

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:addResponse xmlns:ns2="http://soap.com/">
      <return>30</return>
    </ns2:addResponse>
  </S:Body>
</S:Envelope>
```

Link: https://developer.mozilla.org/en-US/docs/Web/XML/Guides/XML_introduction

SoapUI 5.9.1

FileProjectSuiteCaseStepToolsDesktopHelp

EmptySOAPRESTImportSave AllForumTrialPreferencesProxy

Endpoint Explorer

Search Forum

Online Help

Projects

Demo

FreeWSDL

REST Project 1

SOAPEclipse

CalculatorServiceImplPortBinding

add

subtract

StudentProject

Request 1

SoapUI Start Page

Request 1

http://localhost:8080/calculator

Raw XML

```
<?xml version='1.0' encoding='UTF-8'>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <soap:add>
      <arg0>50</arg0>
      <arg1>200</arg1>
    </soap:add>
  </soapenv:Body>
</soapenv:Envelope>
```

Raw XML

```
<?xml version='1.0' encoding='UTF-8'>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:addResponse xmlns:ns2="http://soap.com/">
      <return>250</return>
    </ns2:addResponse>
  </S:Body>
</S:Envelope>
```

Headers (4)Attachments (0)SSL InfoWSS (0)JMS (0)

response time: 22ms (200 bytes)

response time: 1243ms (324 bytes)

AuthHeaders (1)Attachments (0)Representations (1)JMS HeadersJMS Property (0)

response time: 323ms (64897 bytes)

SSL Info (3 certs)

Request Properties

Property	Value
Name	Request 1
Description	
Message Size	278
Encoding	UTF-8
Endpoint	http://localhost:8080

Properties

SoapUI loghttp logjetty logerror logwsrm logmemory log



Package Explorer X

- SOAPCalculator
 - JRE System Library [JavaSE-1.8]
 - src
 - com.soap
 - CalculatorService.java
 - CalculatorServiceImpl.java
 - Publisher.java

CalculatorService.java X CalculatorServiceImpl.java Publisher.java

```
1 package com.soap;  
2  
3 import javax.jws.WebMethod;  
4 import javax.jws.WebService;  
5  
6 @WebService  
7 public interface CalculatorService {  
8  
9     @WebMethod  
10    int add(int a, int b);  
11  
12    @WebMethod  
13    int subtract(int a, int b);  
14 }
```

Outline X Task List

- com.soap
 - CalculatorService
 - add(int, int) : int
 - subtract(int, int)

Problems @ Javadoc Declaration Console X

Publisher [Java Application] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe (28-May-2026, 12:18:05 am elapsed: 0:09:56) [pid: 10304]

SOAP Service Started!



Package Explorer X

SOAPCalculator

- JRE System Library [JavaSE-1.8]
- src
 - com.soap
 - CalculatorService.java
 - CalculatorServiceImpl.java
 - Publisher.java

CalculatorService.java CalculatorServiceImpl.java X Publisher.java

```
1 package com.soap;
2
3 import javax.ws.WebService;
4
5 @WebService(endpointInterface = "com.soap.CalculatorService")
6 public class CalculatorServiceImpl implements CalculatorService {
7
8     @Override
9     public int add(int a, int b) {
10         return a + b;
11     }
12
13     @Override
14     public int subtract(int a, int b) {
15         return a - b;
16     }
17 }
```

Outline X Task List

com.soap

- CalculatorServiceImpl
 - add(int, int) : int
 - subtract(int, int) : int

Problems @ Javadoc Declaration Console X

Publisher [Java Application] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe (28-May-2026, 12:18:05 am elapsed: 0:10:26) [pid: 10304]
SOAP Service Started!



Package Explorer



CalculatorService.java

CalculatorServiceImpl.java

Publisher.java



Outline

Task List

- SOAPCalculator
 - JRE System Library [JavaSE-1.8]
 - src
 - com.soap
 - CalculatorService.java
 - CalculatorServiceImpl.java
 - Publisher.java

```
1 package com.soap;
2
3 import javax.xml.ws.Endpoint;
4
5 public class Publisher {
6
7     public static void main(String[] args) {
8
9         Endpoint.publish(
10             "http://localhost:8080/calculator",
11             new CalculatorServiceImpl()
12         );
13
14         System.out.println("SOAP Service Started!");
15     }
16 }
```

- com.soap
 - Publisher
 - main(String[]): void

Problems @ Javadoc Declaration Console

Publisher [Java Application] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe (28-May-2026, 12:18:05 am elapsed: 0:10:51) [pid: 10304]

SOAP Service Started!

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<!-- Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e. -->
<!-- Generated by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e. -->
<?xml version='1.0' encoding='UTF-8'?>
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/soap/" xmlns:tns="http://soap.com/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://soap.com/" name="CalculatorServiceImplService">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://soap.com/" schemaLocation="http://localhost:8080/calculator?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="add">
    <part name="parameters" element="tns:add"/>
  </message>
  <message name="addResponse">
    <part name="parameters" element="tns:addResponse"/>
  </message>
  <message name="subtract">
    <part name="parameters" element="tns:subtract"/>
  </message>
  <message name="subtractResponse">
    <part name="parameters" element="tns:subtractResponse"/>
  </message>
  <portType name="CalculatorService">
    <operation name="add">
      <input wsam:Action="http://soap.com/CalculatorService/addRequest" message="tns:add"/>
      <output wsam:Action="http://soap.com/CalculatorService/addResponse" message="tns:addResponse"/>
    </operation>
    <operation name="subtract">
      <input wsam:Action="http://soap.com/CalculatorService/subtractRequest" message="tns:subtract"/>
      <output wsam:Action="http://soap.com/CalculatorService/subtractResponse" message="tns:subtractResponse"/>
    </operation>
  </portType>
  <binding name="CalculatorServiceImplPortBinding" type="tns:CalculatorService">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
    <operation name="add">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
    <operation name="subtract">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>
</definitions>
```

Evolution and Emergence of Web Services: Evolution of distributed computing.

Refer PDF File - [unit_1_CORBA.pdf](#)

Application

RMI Client

Stubs

Remote Reference
Layer

Transport Layer

Application

RMI Client

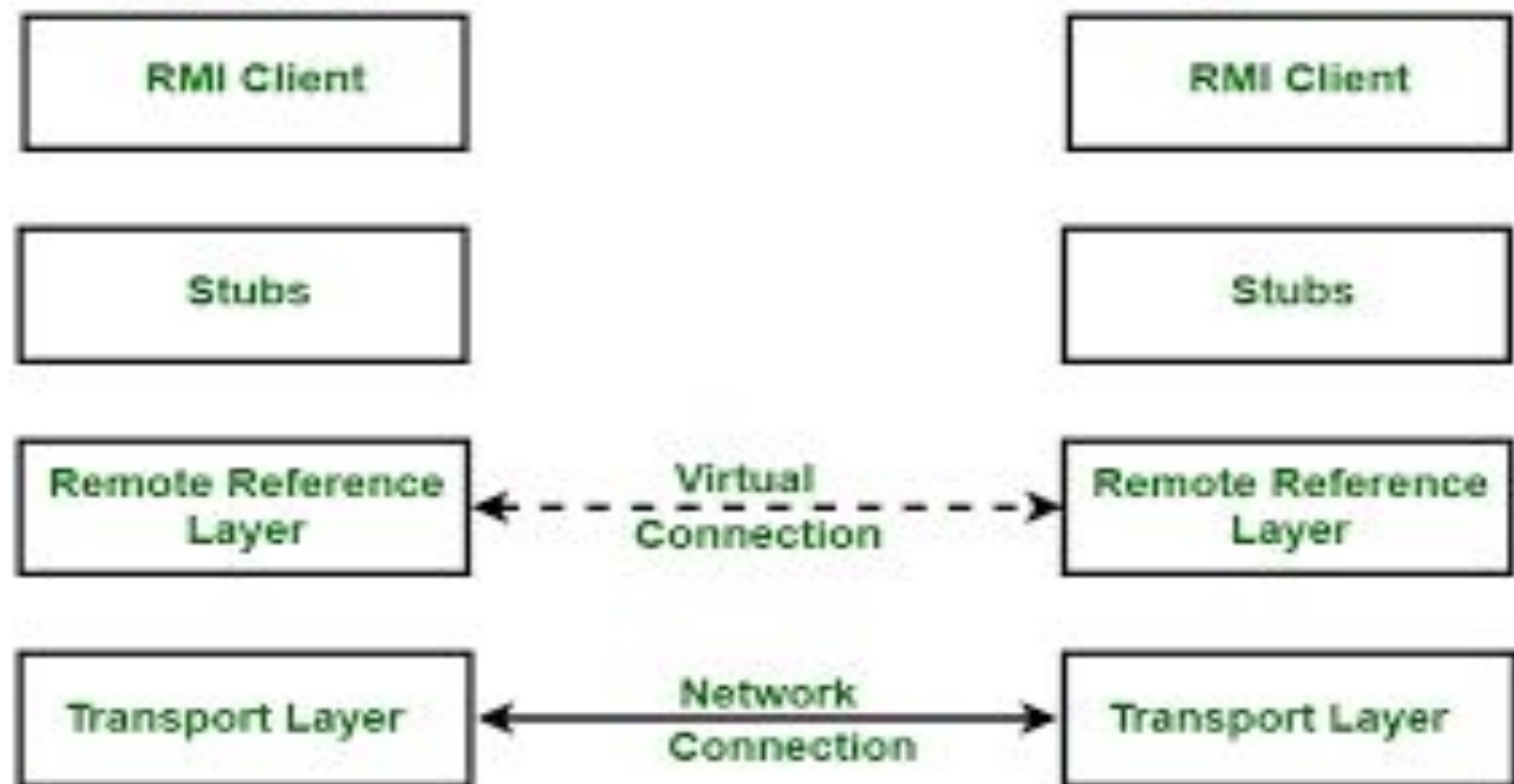
Stubs

Remote Reference
Layer

Transport Layer

Virtual
Connection

Network
Connection



Code Example of client/server Distributed Computing

Client.java

```
import java.io.*;
import java.net.*;
public class Client {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            BufferedReader input = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(
                socket.getOutputStream(), true);
            output.println("Hello Server");
            String response = input.readLine();
            System.out.println("Server Response: " + response);
            socket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Server.java

```
import java.io.*;
import java.net.*;
public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new
ServerSocket(5000);
            System.out.println("Server is
running...");
            System.out.println("Waiting for client
connection...");
            Socket socket = serverSocket.accept();
            System.out.println("Client
connected!");
            BufferedReader input = new
BufferedReader(
                new
InputStreamReader(socket.getInputStream())
            );
            PrintWriter output = new PrintWriter(
                socket.getOutputStream(),
                true);
            String message = input.readLine();
            System.out.println("Message from
Client: " + message);
            output.println("Hello Client, Message
Received!");
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



Package Explorer

- ClientServerExample
 - JRE System Library [JavaSE-1.8]
 - src
 - CRUDRestService
 - RestDemo
 - Servers
 - SOAPCalculator

ResourceRepo... ResourceSer... web.xml CRUDRestSer... Server.java Client.java

```
1 import java.io.*;
2 import java.net.*;
3
4 public class Server {
5
6     public static void main(String[] args) {
7
8         try {
9             ServerSocket serverSocket = new ServerSocket(5000);
10
11             System.out.println("Server is running...");
12             System.out.println("Waiting for client connection...");
13
14             Socket socket = serverSocket.accept();
15
16             System.out.println("Client connected!");
17
18             BufferedReader input = new BufferedReader(
19                 new InputStreamReader(socket.getInputStream()));
20
21             PrintWriter output = new PrintWriter(
22                 socket.getOutputStream(), true);
23
24             String message = input.readLine();
25
26             System.out.println("Message from Client: " + message);
27
28             output.println("Hello Client, Message Received!");
29
30             socket.close();
31             serverSocket.close();
32         }
33     }
34 }
```

Problems @ Javadoc Declaration Console

Server [Java Application] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe (05-Jun-2026, 8:00:08 pm elapsed: 0:00:07) [pid: 36764]
Server is running...
Waiting for client connection...



Package Explorer X

- ClientServerExample
 - JRE System Library [JavaSE-1.8]
 - src
 - CRUDRestService
 - RestDemo
 - Servers
 - SOAPCalculator

ResourceRepo... ResourceSer... web.xml CRUDRestSer... Server.java Client.java X

```
1 import java.io.*;
2 import java.net.*;
3
4 public class Client {
5
6     public static void main(String[] args) {
7
8         try {
9
10             Socket socket = new Socket("localhost", 5000);
11
12             BufferedReader input = new BufferedReader(
13                 new InputStreamReader(socket.getInputStream()));
14
15             PrintWriter output = new PrintWriter(
16                 socket.getOutputStream(), true);
17
18             output.println("Hello Server");
19
20             String response = input.readLine();
21
22             System.out.println("Server Response: " + response);
23
24             socket.close();
25
26         } catch (Exception e) {
27             e.printStackTrace();
28         }
29     }
30 }
```

Problems @ Javadoc Declaration Console X

<terminated> Client [Java Application] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe (05-Jun-2026, 8:00:40 pm – 8:00:40 pm elapsed)
Server Response: Hello Client, Message Received!

Code Example of CORBA

 Note: CORBA support was removed from Java 11 and later. Use **JDK 8** and Eclipse configured with Java 8.

Code Example of RMI

Client.java

```
package my;
import java.rmi.Naming;
public class Client {
    public static void main(String[] args) {
        try {
            XMLService service =
                (XMLService) Naming.lookup(
                    "rmi://localhost/XMLService");
            String xml = service.getStudentXML();
            System.out.println("Received XML:");
            System.out.println(xml);
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```


Server.java

```
package my;

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class Server {

    public static void main(String[] args) {

        try {
            LocateRegistry.createRegistry(1099);

            XMLService service = new XMLServiceImpl();

            Naming.rebind("rmi://localhost/XMLService", service);

            System.out.println("RMI XML Server Started...");
        }
        catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

XMLService.java (Interface)

```
package my;
```

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
```

```
public interface XMLService extends Remote {
```

```
    String getStudentXML() throws RemoteException;  
}
```

```
XMLServiceImpl.java (Class)
package my;

import java.rmi.RemoteException;
import
java.rmi.server.UnicastRemoteObject;

public class XMLServiceImpl extends
UnicastRemoteObject implements
XMLService {

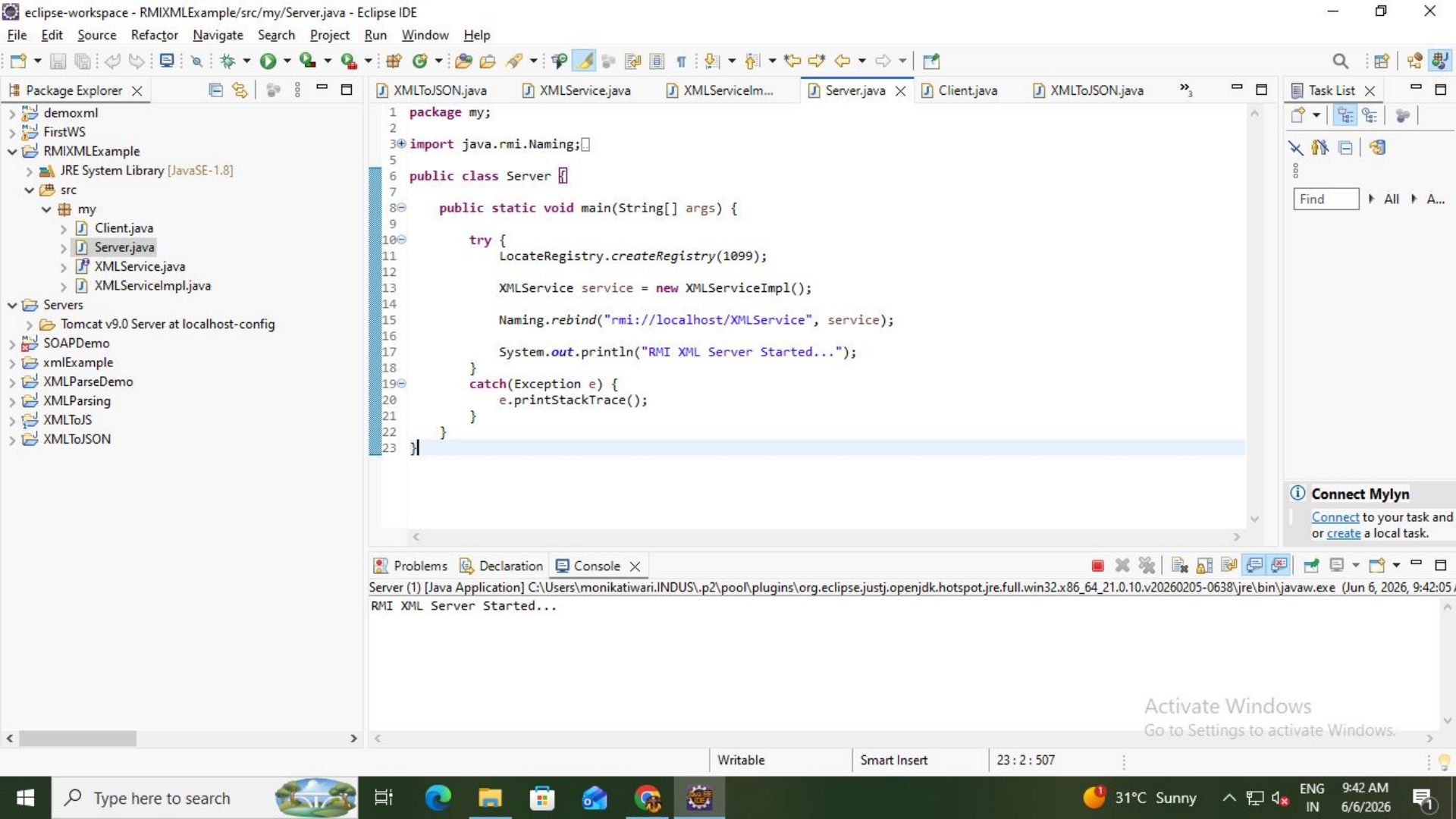
    private static final long
serialVersionUID = 1L;

    protected XMLServiceImpl() throws
RemoteException {
        super();
    }
}
```

```
@Override
    public String getStudentXML()
throws RemoteException {

        String xml =
            "<?xml version=\"1.0\"?>" +
            "<student>" +
            "<id>101</id>" +
            "<name>Monika</name>" +
            "<course>MCA</course>" +
            "</student>";

        return xml;
    }
}
```



Package Explorer

- demoxml
- FirstWS
- RMIXMLExample
 - JRE System Library [JavaSE-1.8]
 - src
 - my
 - Client.java
 - Server.java
 - XMLService.java
 - XMLServiceImpl.java
 - Servers
 - Tomcat v9.0 Server at localhost-config
 - SOAPDemo
 - xmlExample
 - XMLParseDemo
 - XMLParsing
 - XMLToJS
 - XMLToJSON

XMLToJSON.java

XMLService.java

XMLServiceImpl...

Server.java

Client.java

XMLToJSON.java

Task List

```

1 package my;
2
3 import java.rmi.Naming;
4
5 public class Server {
6
7
8     public static void main(String[] args) {
9
10         try {
11             LocateRegistry.createRegistry(1099);
12
13             XMLService service = new XMLServiceImpl();
14
15             Naming.rebind("rmi://localhost/XMLService", service);
16
17             System.out.println("RMI XML Server Started...");
18         }
19         catch (Exception e) {
20             e.printStackTrace();
21         }
22     }
23 }

```

Find

All

A...

Find

All

A...

Find

All

A...

Find

All

A...

Find

All

A...

Find

All

A...

Find

All

A...

Find

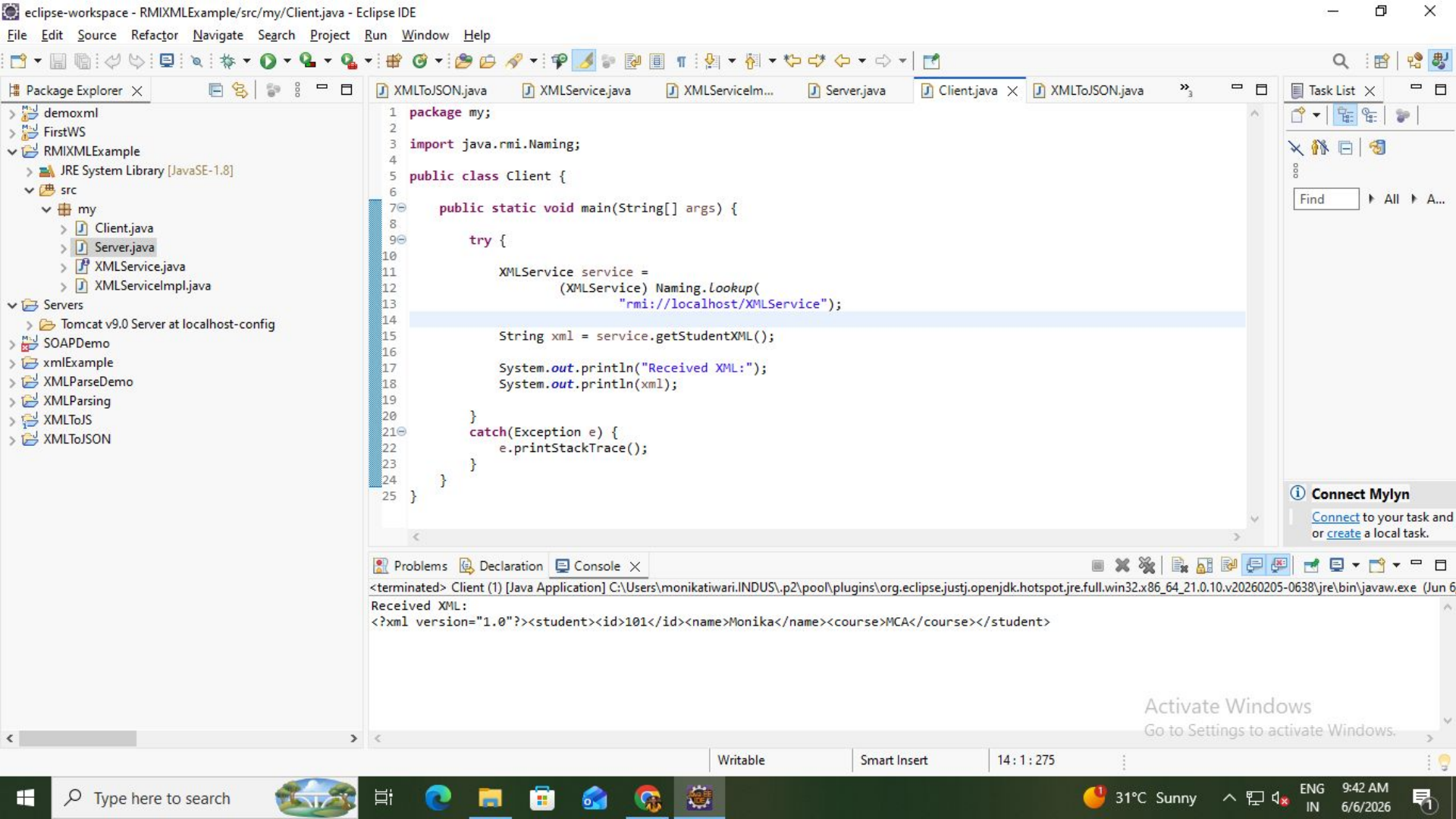
All

A...

Find

All

A...



XML to JSON file Convert

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import org.json.JSONObject;
import org.json.XML;
public class XMLToJSON {
    public static void main(String[] args) {
        try {
            // Read XML file
            BufferedReader br = new BufferedReader(
                new FileReader("student.xml"));
            String line;
            StringBuilder xmlContent = new
            StringBuilder();
            while ((line = br.readLine()) != null) {
                xmlContent.append(line);
            }
            br.close();
```

```
            // Convert XML to JSON
            JSONObject jsonObject =
            XML.toJSONObject(xmlContent.toString());
            // Generate JSON file
            FileWriter file =
                new FileWriter("student.json");
            file.write(jsonObject.toString(4));
            file.close();
            System.out.println(
                "JSON file generated
            successfully!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Student.XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<student>
```

```
  <id>101</id>
```

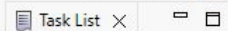
```
  <name>Monika</name>
```

```
  <course>MCA</course>
```

```
  <email>monika@gmail.com</email>
```

```
</student>
```

- Run the java file
- Refresh project > student.json will appear in project folder



11

 **Connect Mylyn**

Connect to your task and

or [create](#) a local task.

full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Jun 6

Go to Settings to activate Windows.

